

PLANO PARA DESENVOLVIMENTO SEGURO

Cidade Virtual - PI - III

Avaliação do Módulo 3 - Boas Práticas em Serviços Web

Maio de 2026

Nome Completo:	Diego Serafim de Sousa
Disciplina:	Projeto Integrador de Tecnologia da Informação III
Semestre:	2026/1
Curso:	Tecnologia da Informação
Público-alvo:	Moradores, comerciantes e autônomos.
Local de realização:	Cabanagem, Belém, Pará

1 TEMA DA AÇÃO

PLANO PARA O DESENVOLVIMENTO SEGURO DO PROJETO CIDADE VIRTUAL

2 RESUMO

Este documento apresenta o Plano para o Desenvolvimento Seguro do projeto Cidade Virtual, uma plataforma web desenvolvida para conectar moradores, comerciantes e autônomos do bairro e da cidade. Foram identificados os principais riscos de segurança associados ao armazenamento de dados pessoais e à autenticação de usuários. O plano estabelece 18 práticas de segurança distribuídas em seis camadas (infraestrutura, aplicação, API, log/auditing, ferramentas e erros/exceções), alinhadas ao OWASP Top 10. São propostas políticas de governança para revisão contínua e um cronograma de implantação de 5 semanas. O objetivo é garantir a confidencialidade, integridade e disponibilidade dos dados da comunidade, mitigando riscos de vulnerabilidades comuns em aplicações web.

Palavras-chave: Desenvolvimento seguro. Segurança de software. OWASP. Cidade Virtual. Tecnologia social.

3 INTRODUÇÃO

O projeto Cidade Virtual vem sendo desenvolvido desde 2022 pelo estudante Diego Serafim de Sousa, sendo continuamente aprimorado no âmbito das disciplinas do Projeto Integrador da UFMS. Trata-se de uma plataforma web com pilha MERN (MongoDB, Express.js, React.js, Node.js) que visa conectar moradores, comerciantes locais e trabalhadores autônomos da comunidade, promovendo o fortalecimento da economia local e a integração comunitária.

3.1 Características da aplicação

A aplicação armazena e processa informações sensíveis dos usuários, incluindo:

Tabela 1 – Tipos de dados armazenados e riscos associados

Tipo de dado	Exemplos	Risco associado
Dados cadastrais	Nome, email, telefone, endereço	Exposição de identidade
Credenciais de acesso	Senhas (hash armazenado)	Acesso não autorizado
Dados de comércio	CNPJ, horários, categoria, avaliações	Impacto no negócio local
Dados de interação	Avaliações, comentários, eventos	Reputação e integridade

Fonte: Elaborado pelo autor (2026)

3.2 Impacto dos aspectos de segurança no negócio

A ausência de práticas adequadas de desenvolvimento seguro pode impactar negativamente o projeto de extensão nas seguintes dimensões:

- **Confiança da comunidade:** Um sistema com vulnerabilidades (ex.: vazamento de dados) faria com que os comerciantes e moradores perdessem a confiança na plataforma, inviabilizando o objetivo de extensão.
- **Integridade das informações:** A manipulação indevida de avaliações ou a exclusão não autorizada de comércios prejudicaria a economia local.
- **Disponibilidade:** Ataques de negação de serviço (DDOS) poderiam tornar a plataforma inacessível exatamente quando a comunidade mais precisa.
- **Responsabilidade legal:** O armazenamento inadequado de dados pessoais (especialmente após a LGPD) expõe o projeto a riscos jurídicos.

3.3 Motivação para implantação do plano

A adoção de um plano de desenvolvimento seguro é motivada por:

1. **Proteção dos usuários da extensão:** A comunidade merece um sistema que respeite a sua privacidade e proteja seus dados pessoais.
2. **Alinhamento ao OWASP Top 10:** As vulnerabilidades mais comuns em aplicações web (injeção, XSS, autenticação quebrada, etc.) podem ser mitigadas com práticas sistemáticas.
3. **Desenvolvimento individual consciente:** Como o projeto é desenvolvido por uma única pessoa, a disciplina de seguir um plano formal de segurança é ainda mais crítica para evitar pontos cegos.

- 4. Custo da correção tardia:** Identificar e corrigir vulnerabilidades em produção é significativamente mais caro do que prevenir durante o desenvolvimento.

4 OBJETIVOS

4.1 Objetivo geral

Estabelecer um conjunto de práticas sistemáticas de desenvolvimento seguro para o projeto Cidade Virtual, abrangendo diferentes camadas do processo, visando garantir a confidencialidade, integridade e disponibilidade dos dados da comunidade.

4.2 Objetivos específicos

Tabela 2 – Objetivos específicos e benefícios esperados para o desenvolvimento seguro

ID	Objetivo	Benefício esperado
OE1	Implementar controles de segurança nas camadas de infraestrutura, aplicação, API, logs, ferramentas e tratamento de erros	Redução da superfície de ataque e mitigação das vulnerabilidades do OWASP Top 10
OE2	Adotar práticas de autenticação robusta (JWT com expiração, bcrypt, rate limiting)	Prevenção contra acesso não autorizado e força bruta
OE3	Estabelecer política de governança com revisões periódicas e uso de ferramentas automatizadas (OWASP ZAP, npm audit)	Deteção precoce de vulnerabilidades e melhoria contínua
OE4	Capacitar o desenvolvedor (único responsável) em princípios de segurança	Conscientização e redução de erros humanos
OE5	Garantir a rastreabilidade de ações sensíveis (logs de acesso e operações)	Capacidade de auditoria e resposta a incidentes

Fonte: Elaborado pelo autor (2026)

5 PRÁTICAS DE SEGURANÇA

Nas tabelas abaixo serão apresentadas 18 práticas de segurança organizadas por camadas, alinhadas às recomendações do OWASP Top 10 e à OWASP Cheat Sheet Series. Cada prática contém título, descrição, motivação e referência.

5.1 Camada de Infraestrutura

Tabela 3 – Práticas de segurança aplicadas à camada de infraestrutura

ID	Prática	Descrição	Motivação	Referência
INF01	HTTPS obrigatório	Todo tráfego entre cliente e servidor deve ser criptografado com TLS 1.2 ou superior. No ambiente Vercel, ativar redirecionamento automático de HTTP para HTTPS	Proteger dados em trânsito contra interceptação (ataques Man-in-the-Middle)	OWASP A2 (Falhas Criptográficas)
INF02	Isolamento de ambientes	Manter ambientes separados para o desenvolvimento, homologação e produção. Utilizar variáveis de ambiente distintas (.env.development, .env.production)	Evita que configurações de desenvolvimento (ex.: debug ativo, chaves fracas) cheguem à produção	OWASP A5 (Configuração Incorreta)
INF03	Backups criptografados	Realizar backups diários do banco de dados MongoDB (Atlas), armazenando cópias criptografadas em local separado	Garante disponibilidade e integridade dos dados em caso de ataque de ransomware ou falha catastrófica	Tríade CID (Disponibilidade)

Fonte: Elaborado pelo autor (2026)

5.2 Camada de Aplicação

Tabela 4 – Práticas de segurança aplicadas à camada de aplicação

ID	Prática	Descrição	Motivação	Referência
APP01	Hashing de senhas com bcrypt	Armazenar senhas utilizando bcrypt com fator de custo 10. Nunca armazenar senhas em texto plano.	Protege credenciais em caso de vazamento do banco de dados	OWASP A2 (Falhas Criptográficas)
APP02	Autenticação JWT com expiração	Utilizar JSON Web Token com tempo de expiração curto (1 hora para acess token, 7 dias para refresh token)	Limitar a janela de exposição em caso de token roubado	OWASP A7 (Falhas de Identificação e Autenticação)
APP03	Política de	Exigir senhas com no	Reduzir a	OWASP A7

	senha forte	mínimo 8 caracteres, incluindo letras maiúsculas, minúsculas, números e símbolos. Implementar verificação de senhas comuns (rockyou.txt)	probabilidade de sucesso em ataques de força bruta e dicionário.	(Falhas de Identificação e Autenticação)
APP04	Rate limiting (Limitação de requisições)	Limitar tentativas de login para 5 tentativas em 15 minutos, com bloqueio temporário. Aplicar também em endpoints de cadastro e recuperação de senha	Mitigar ataques de força bruta e DDoS de baixa intensidade	OWASP A7 (Falhas de Identificação e Autenticação)
APP05	Sanitização de saída	Escapar caracteres especiais (HTML, JavaScript) em qualquer dado exibido na interface para evitar XSS. Utilizar bibliotecas como DOMPurify no front-end	Prevenir ataques de Cross-Site Scripting (XSS)	OWASP A3 (Injeção)
APP06	Validação de entrada em todas as camadas	Validar e sanitizar todos os dados recebidos (Body, query, params) no back-end, mesmo que já validados no front-end. Utilizar express-validator ou joi	Impede injeção de comandos (NoSQL, SQL) e dados malformados	OWASP A3 (Injeção)

Fonte: Elaborado pelo autor (2026)

5.3 Camada de API

Tabela 5 – Práticas de segurança aplicadas à camada de API

ID	Prática	Descrição	Motivação	Referência
API01	Verificação de autorização por endpoint	Cada endpoint deve verificar se o usuário autenticado tem permissão para acessar o recurso (ex.: morador não pode editar comércio de outro)	Prevenir Acesso Quebrado (IDOR - Insecure Direct Object References)	OWASP A1 (Controle de Acesso Quebrado)
API02	CORS restrito	Configurar CORS para permitir apenas as origens necessárias (ex.: domínio do front-end na Vercel). Nunca usar Access-Control-Allow-Origin: * em produção	Evita que sites maliciosos façam requisições não autorizadas em nome do usuário	OWASP A5 (Configuração Incorreta)

API03	Versionamento da API	Utilizar prefixo /v1/ nas URLs. Para mudanças que quebram a compatibilidade, criar /v2/ e manter versão antiga por período de transição	Permite evolução segura da API sem quebrar clientes existentes	Boa prática REST
API04	Limitação de tamanho de payload	Configurar limite de tamanho para requisições (ex.: 1MB para JSON)	Previne ataques de negação de serviço por envio de payloads gigantescos	OWASP A8 (Falhas de Integridade)

Fonte: Elaborado pelo autor (2026)

5.4 Camada e Log e Auditing

Tabela 6 – Práticas de segurança aplicadas à camada de log e auditoria

ID	Prática	Descrição	Motivação	Referência
LOG01	Registro de ações críticas	Registrar em log todas as operações sensíveis: login, logout, cadastro de comércio, exclusão de avaliação, alteração de perfil. Incluir timestamp, ID do usuário, ação e IP	Permite auditoria e investigação de incidentes	OWASP A9 (Falhas de Monitoramento e Log)
LOG02	Proteção de logs	Armazenar logs em local separado (ex.: serviço de logs) com permissões restritas. Logs não devem ser acessíveis pela aplicação	Impede que um invasor apague evidências de ataque	OWASP A9 (Falhas de Monitoramento e Log)
LOG03	Não registrar dados sensíveis	Evitar registrar senhas, tokens JWT, ou dados pessoais completos (ex.: CPF). Se necessário, mascarar (ex.: últimos 4 dígitos)	Conformidade com a LGPD e redução de risco em caso de vazamento de logs	LGPD - Princípio da Necessidade

Fonte: Elaborado pelo autor (2026)

5.5 Camada de Ferramentas

Tabela 7 – Práticas de segurança aplicadas ao uso de ferramentas

ID	Prática	Descrição	Motivação	Referência
FERR01	Verificação de dependência	Executar npm audit semanalmente e antes de cada commit. Corrigir	Identificar componentes com CVEs conhecidas	OWASP A6 (Componentes Vulneráveis)

	(npm audit)	vulnerabilidades críticas imediatamente	(OWASP A6)	
FERR02	Escaneamento automático com OWASP ZAP	Configurar OWASP ZAP para escaneamento passivo e ativo periódico (mensal), Além de execução antes de releases	Detectar vulnerabilidades como injeção, XSS e configuração incorreta	OWASP A3, A5
FERR03	Análise estática (ESLint com segurança)	Utilizar ESLint com plugins de segurança (eslint-plugin-security) para detectar padrões inseguros no código	Capturar problemas como eval(), concatenação de SQL, uso de Math.random() para criptografia	Boa prática de DevSecOps
FERR04	Versionamento seguro (Git)	Nunca commitar arquivos .env, chaves privadas, certificados ou credenciais. Utilizar .gitignore rigoroso	Evitar exposição acidental de segredos em repositórios públicos	OWASP A5 (Configuração Incorreta)

Fonte: Elaborado pelo autor (2026)

5.6 Camada de Erros e Exceções

Tabela 8 – Práticas de segurança aplicadas ao tratamento de erros e exceções

ID	Prática	Descrição	Motivação	Referência
ERR01	Mensagens de erro genéricas	Em produção, exibir mensagens genéricas ('Ocorreu um erro interno') ao usuário. Detalhes (stack trace) devem ir apenas para logs	Evita que informações internas (versões, caminhos, estrutura de banco) sejam expostas	OWASP A5 (Configuração Incorreta)
ERR02	Tratamento global de exceções	Implementar middleware de tratamento de erros no Express que captura exceções não tratadas e retorna resposta padronizada	Evita que erros não tratados derrubem a aplicação ou exponham detalhes sensíveis	Boa prática de segurança

Fonte: Elaborado pelo autor (2026)

6 IMPLANTAÇÃO E GOVERNANÇA

6.1 Meios para implantação das práticas

Para a implantação das práticas listadas na seção 5, serão utilizados os recursos e abordagens:

Tabela 9 – Recursos e finalidades para implantação das práticas de segurança

Recurso	Finalidade
Ambiente de desenvolvimento local (Node.js MongoDB)	Teste seguro antes da homologação
Vercel (hospedagem)	Configuração de HTTPS, variáveis de ambiente, CORS
MongoDB Atlas	Backups automáticos, criptografia em repouso
GitHub Actions	Automação de <code>npm audit</code> e execução de OWASP ZAP básico
OWASP ZAP (instalação local)	Escaneamento detalhado de vulnerabilidades
ESLint, plugin security	Análise estática no pré-commit (husky)

Fonte: Elaborado pelo autor (2026)

6.2 Governança e revisão contínua

A governança das práticas de segurança será conduzida pelo próprio desenvolvedor (único responsável), seguindo as seguintes políticas:

6.2.1 Política de revisão periódica

Tabela 10 – Cronograma de atividades e responsabilidades para revisão periódica de segurança

Periodicidade	Atividade	Responsável
Semanal	Executar <code>npm audit</code> e corrigir vulnerabilidades críticas	Diego (desenvolvedor)
Quinzenal	Revisão do checklist OWASP ASVS (nível básico)	Diego
Mensal	Escaneamento completo com OWASP ZAP (modo ativo)	Diego
A cada release (aprox. 2 meses)	Revisão geral de segurança antes da implantação	Diego

Fonte: Elaborado pelo autor (2026)

6.2.2 Política de resposta a incidentes

Em caso de identificação de vulnerabilidade crítica ou suspeita de invasão:

1. **Isolamento imediato:** Desabilitar funcionalidade afetada ou colocar site em manutenção.
2. **Análise:** Verificar logs para identificar escopo e impacto.
3. **Correção:** Implementar patch e testar em ambiente isolado.
4. **Comunicação:** Informar os usuários afetados (comerciantes, moradores) se dados pessoais foram comprometidos.
5. **Retrospectiva:** Documentar a causa raiz e ajustar práticas para evitar recorrência.

6.2.3 Política de atualização de dependências

- Toda semana, executar npm outdated e planejar atualizações de dependências.
- Priorizar atualizações de segurança (identificadas por npm audit).
- Após atualização, executar bateria de testes (Jest, Supertest) para validar compatibilidade.

7 CRONOGRAMA

O cronograma de implantação das práticas de segurança é estruturado em 5 semanas, com esforço estimado total de 32 horas, respeitando o caráter individual do projeto.

Tabela 11 – Cronograma de implantação, esforço estimado e entregáveis das práticas de segurança

Semana	Práticas prioritárias	Camada	Esforço	Entregáveis
Semana 1 (24-30/05/2026)	INF01, INF02, APP02, APP04, ERR01, ERR02	Infraestrutura, Aplicações, Erros	8h	HTTPS ativo; autenticação JWT implementada; rate limiting no login; middleware global de erros
Semana 2 (31-05/06/2026)	APP01, APP03, APP06	Aplicação	8h	Bcrypt nas senhas; política de senha forte; validação de entrada com express-validator
Semana 3 (06-13/06/2026)	API01, API02, API03, API04	API	6h	Controle de autorização por endpoint; CORS restrito; versionamento /v1/; limite de payload

Semana 4 (14-20/06/2026)	LOG01, LOG02, LOG03, FERR04	Log, Ferramentas	6h	Logs estruturados; proteção de logs; .gitignore rigoroso
Semana 5 (21-27/06/2026)	APP05, FERR01, FERR02, FERR03	Aplicação, Ferramentas	4h	Sanitização de saída; npm audit integrado ao CI; OWASP ZAP configurado; ESLint security

Fonte: Elaborado pelo autor (2026)

REFERÊNCIAS

OWASP FOUNDATION. **OWASP Top Ten 2021**. Disponível: <https://link.ufms.br/x0AMA>. Acesso em: 12 fev. 2025.

OWASP FOUNDATION. **OWASP Cheat Sheet Series**. Disponível em: <https://cheatsheetseries.owasp.org/>. Acesso em: 21 mai. 2026.

OWASP FOUNDATION. **Security Fundamentals**. Disponível: <https://link.ufms.br/nbl2h>. Acesso em: 12 fev. 2025.

NETO, Tarcizio Vieira. **Modelo de processo para desenvolvimento de aplicações seguras**. Disponível em: <https://link.ufms.br/nsb26>. Acesso em: 22 nov. 2024.

PRESSMAN, Roger S. **Engenharia de Software: uma abordagem profissional**. 8. ed. Porto Alegre: AMGH, 2016. Capítulo 14 (Segurança de Software).

SOMMERVILLE, Ian. **Engenharia de software**. 10. ed. São Paulo: Pearson, 2018. Capítulo 12 (Segurança e Engenharia de Software).

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6023**: Informação e documentação - Referências - Elaboração. Rio de Janeiro: ABNT, 2018.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 14724**: Informação e documentação - Trabalhos acadêmicos - Apresentação. Rio de Janeiro: ABNT, 2011.